

Lab 2: Sequential Logic & Memory Hierarchy

Name: Sukhvinder Kaur

Roll No: 2501010272

Q-1. 1-bit DFF-based register (Bit chip)

1. Boolean Expression

The Bit chip stores one bit. A DFF only remembers its input from the previous clock cycle, so a Mux is placed in front of it. If load = 1 the new input is stored, otherwise the old output is fed back.

DFF input: $d = (load \cdot in) + (load' \cdot out(t-1))$

Output: $out(t) = d(t-1)$

Combined: $out(t) = (load(t-1) \cdot in(t-1)) + (load(t-1)' \cdot out(t-1))$

2. Truth Table

Next-state table (Q = present output, Q+ = output after the next clock cycle):

load	in	Q (present out)	Q+ (next out)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

When load = 0 the bit keeps its old value ($Q+ = Q$). When load = 1 the bit takes the value of in ($Q+ = in$).

3. Block Diagram

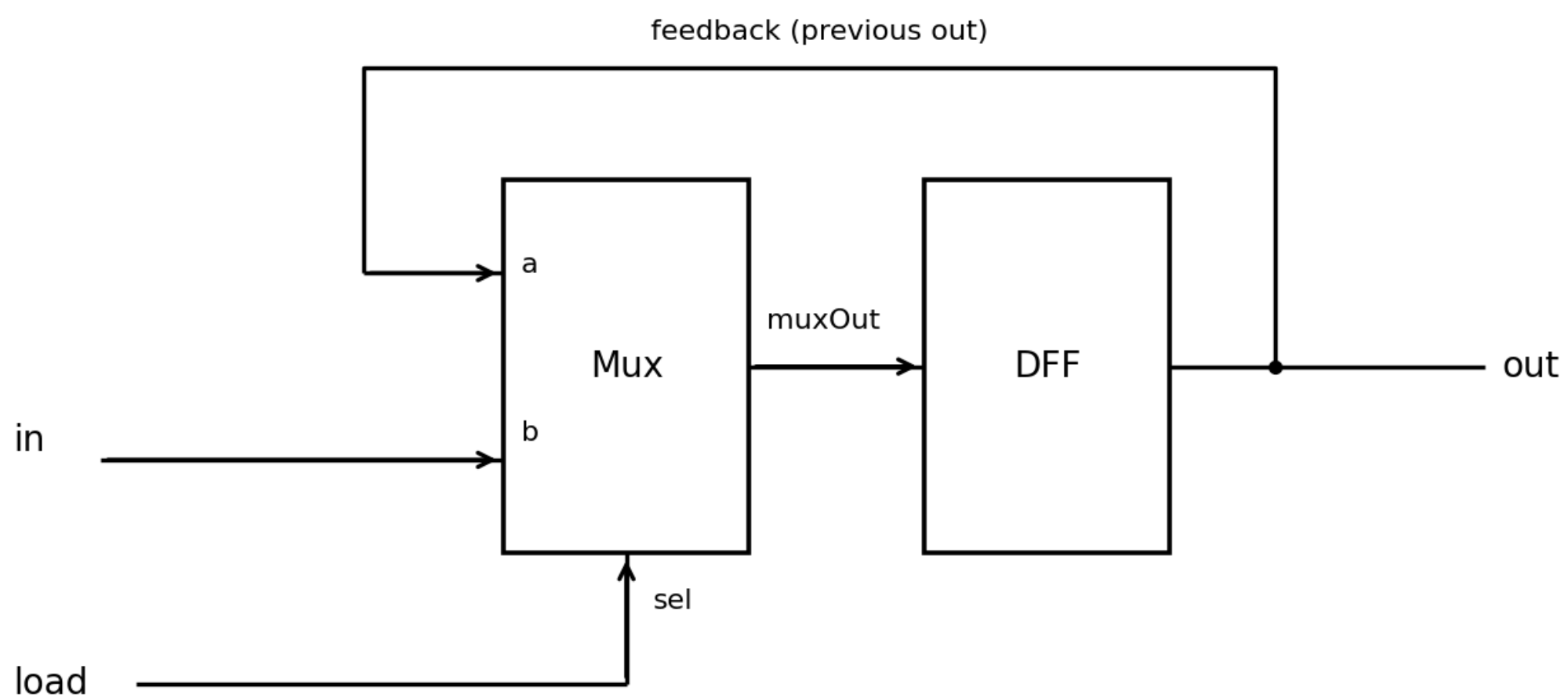


Fig. 1: Bit chip (Mux + DFF with feedback)

4. HDL Code

```

/** Bit.hdl
 * 1-bit register.
 * If load[t-1]: out[t] = in[t-1]
 * else          out[t] = out[t-1]
 */
CHIP Bit {
    IN in, load;
    OUT out;

    PARTS:
    Mux(a=feedback, b=in, sel=load, out=muxOut);
    DFF(in=muxOut, out=out, out=feedback);
}

```

Test Script (Bit.tst)

```

load Bit.hdl,
output-file Bit.out,
compare-to Bit.cmp,
output-list time%S1.4.1 in%B2.1.2 load%B2.1.2 out%B2.1.2;

```

```

set in 0, set load 0, tick, output; tock, output;
set in 1, set load 1, tick, output; tock, output;
set in 0, set load 0, tick, output; tock, output;
set in 0, set load 1, tick, output; tock, output;
set in 1, set load 0, tick, output; tock, output;
set in 1, set load 1, tick, output; tock, output;

```

Clock Cycle Trace (tick-tock behaviour)

In the tick phase (clock goes 0 to 1) the DFF still shows the old value. The new value appears only after the tock phase (clock goes 1 to 0), so the output is always one clock cycle behind the input.

Cycle	Time	in	load	out	Remark
1	1	0	0	0	Hold (initial value 0)
	1+	0	0	0	
2	2	1	1	0	Write 1: output still old value in tick
	2+	1	1	1	New value appears after tock
3	3	0	0	1	load = 0, so value 1 is held
	3+	0	0	1	
4	4	0	1	1	Write 0: output still 1 in tick
	4+	0	1	0	New value 0 appears after tock
5	5	1	0	0	load = 0, in is ignored
	5+	1	0	0	
6	6	1	1	0	Write 1
	6+	1	1	1	Output becomes 1 after tock

Read operation: the stored value is always available on out (reading needs no control signal). Write operation: it happens only when load = 1, and the stored value changes at the end of that clock cycle.

Q-2. 16-bit Register chip

1. Boolean Expression

The 16-bit Register is made of sixteen Bit chips that share the same load signal. For every bit i from 0 to 15:

$$\text{out}[i](t) = (\text{load}(t-1) \cdot \text{in}[i](t-1)) + (\text{load}(t-1)' \cdot \text{out}[i](t-1))$$

So if $\text{load} = 1$, $\text{out} = \text{in}$ (after the clock edge). If $\text{load} = 0$, out keeps its previous value.

2. Truth Table

The truth table of one bit slice is the same as the Bit chip, and all 16 slices work in the same way:

load	in[i]	out[i] (present)	out[i] (next)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

3. Block Diagram

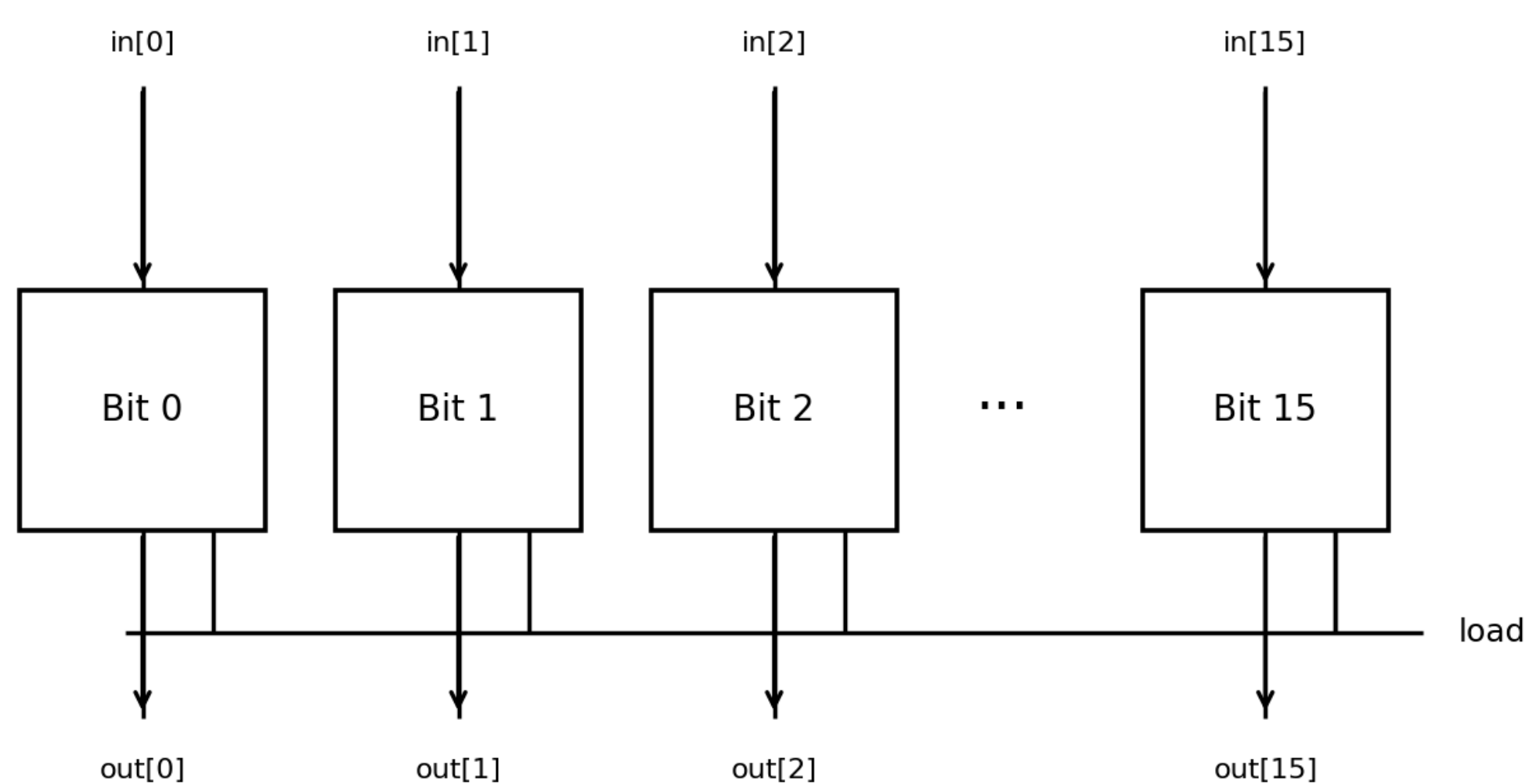


Fig. 2: 16-bit Register built from 16 Bit chips

4. HDL Code

```
/** Register.hdl
 * 16-bit register.
 * If load[t]==1 then out[t+1] = in[t]
 * else out does not change.
 */
CHIP Register {
    IN in[16], load;
    OUT out[16];

    PARTS:
    Bit(in=in[0], load=load, out=out[0]);
    Bit(in=in[1], load=load, out=out[1]);
    Bit(in=in[2], load=load, out=out[2]);
    Bit(in=in[3], load=load, out=out[3]);
    Bit(in=in[4], load=load, out=out[4]);
    Bit(in=in[5], load=load, out=out[5]);
    Bit(in=in[6], load=load, out=out[6]);
    Bit(in=in[7], load=load, out=out[7]);
    Bit(in=in[8], load=load, out=out[8]);
    Bit(in=in[9], load=load, out=out[9]);
    Bit(in=in[10], load=load, out=out[10]);
    Bit(in=in[11], load=load, out=out[11]);
    Bit(in=in[12], load=load, out=out[12]);
    Bit(in=in[13], load=load, out=out[13]);
    Bit(in=in[14], load=load, out=out[14]);
    Bit(in=in[15], load=load, out=out[15]);
}
```

Test Script (Register.tst)

```
load Register.hdl,
output-file Register.out,
compare-to Register.cmp,
output-list time%S1.4.1 in%B1.16.1 load%B2.1.2 out%B1.16.1;
```

```
set in %B1010101010101010, set load 1, tick, output; tock, output;
set in %B0000111100001111, set load 0, tick, output; tock, output;
set in %B1111111111111111, set load 1, tick, output; tock, output;
```

Read / Write Operations for Three Sample Inputs

Sample	in	load	Operation
1	1010101010101010	1	Write
2	0000111100001111	0	Read only (no write)
3	1111111111111111	1	Write

Simulation Output (Register.out)

time	in	load	out
0+	1010101010101010	1	0000000000000000
1	1010101010101010	1	1010101010101010
1+	0000111100001111	0	1010101010101010
2	0000111100001111	0	1010101010101010
2+	1111111111111111	1	1010101010101010

| 3 | 1111111111111111 | 1 | 1111111111111111 |

Explanation: In sample 1, load = 1 so 1010101010101010 is stored, and it appears on out after the tock. In sample 2, load = 0, so the new input is ignored and out still shows 1010101010101010 (a read of the stored value). In sample 3, load = 1 again, so out is updated to 1111111111111111.

Q-3. 8-register Register File (RAM8)

1. Boolean Expression

RAM8 has eight 16-bit registers (R0 to R7) selected by a 3-bit address $a_2 a_1 a_0$. The load signal is sent only to the register that is addressed (DMux8Way), and the output is taken from the addressed register (Mux8Way16).

Load lines (DMux8Way):

$$\begin{aligned} \text{load0} &= \text{load} \cdot a_2' \cdot a_1' \cdot a_0' \\ \text{load1} &= \text{load} \cdot a_2' \cdot a_1' \cdot a_0 \\ \text{load2} &= \text{load} \cdot a_2' \cdot a_1 \cdot a_0' \\ \text{load3} &= \text{load} \cdot a_2' \cdot a_1 \cdot a_0 \\ \text{load4} &= \text{load} \cdot a_2 \cdot a_1' \cdot a_0' \\ \text{load5} &= \text{load} \cdot a_2 \cdot a_1' \cdot a_0 \\ \text{load6} &= \text{load} \cdot a_2 \cdot a_1 \cdot a_0' \\ \text{load7} &= \text{load} \cdot a_2 \cdot a_1 \cdot a_0 \end{aligned}$$

Output (Mux8Way16):

$$\begin{aligned} \text{out} &= (\text{R0} \cdot a_2'a_1'a_0') + (\text{R1} \cdot a_2'a_1'a_0) + (\text{R2} \cdot a_2'a_1a_0') + (\text{R3} \cdot a_2'a_1a_0) + (\text{R4} \cdot a_2a_1'a_0') \\ &+ (\text{R5} \cdot a_2a_1'a_0) + (\text{R6} \cdot a_2a_1a_0') + (\text{R7} \cdot a_2a_1a_0) \end{aligned}$$

2. Truth Table

DMux8Way (which register gets the load signal):

load	a2	a1	a0	Register loaded
1	0	0	0	R0
1	0	0	1	R1
1	0	1	0	R2
1	0	1	1	R3
1	1	0	0	R4
1	1	0	1	R5
1	1	1	0	R6
1	1	1	1	R7
0	x	x	x	None (no write)

Mux8Way16 (which register is read): out = R0 for address 000, R1 for 001, R2 for 010, R3 for 011, R4 for 100, R5 for 101, R6 for 110 and R7 for 111.

3. Block Diagram

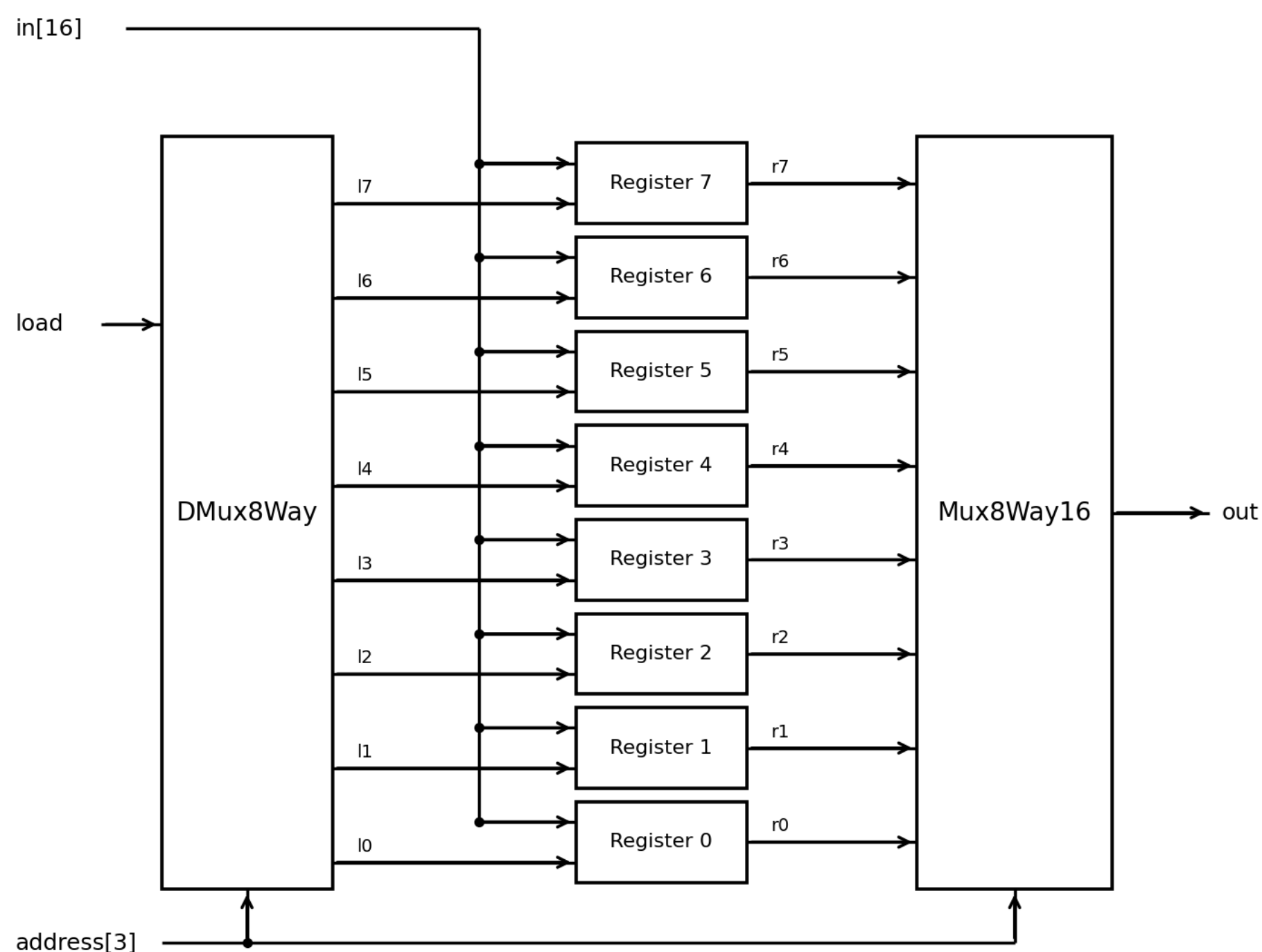


Fig. 3: RAM8 (DMux8Way + 8 Registers + Mux8Way16)

4. HDL Code

```

/** RAM8.hdl
 * Memory of 8 registers, each 16 bits wide.
 * out(t) = RAM[address(t)](t)
 * If load(t-1): RAM[address(t-1)](t) = in(t-1)
 */
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
        DMux8Way(in=load, sel=address, a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6,
h=l7);

        Register(in=in, load=l0, out=r0);
        Register(in=in, load=l1, out=r1);
        Register(in=in, load=l2, out=r2);
        Register(in=in, load=l3, out=r3);
        Register(in=in, load=l4, out=r4);
        Register(in=in, load=l5, out=r5);
        Register(in=in, load=l6, out=r6);
        Register(in=in, load=l7, out=r7);

        Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address,
out=out);
}

```

Read / Write Trace (4 different addresses)

All registers are 0 at the start. Values are written in decimal for easy reading.

Step	Operation	address	in	load	out
1	Write	000 (R0)	11	1	11
2	Write	011 (R3)	25	1	25
3	Write	101 (R5)	100	1	100
4	Write	111 (R7)	255	1	255
5	Read	011 (R3)	0	0	25
6	Read (try write, load=0)	101 (R5)	999	0	100
7	Read	001 (R1)	0	0	0

Register States After Each Write

After	R0	R1	R2	R3	R4	R5	R6	R7
Initial	0	0	0	0	0	0	0	0
Write 1 (R0 = 11)	11	0	0	0	0	0	0	0
Write 2 (R3 = 25)	11	0	0	25	0	0	0	0
Write 3 (R5 = 100)	11	0	0	25	0	100	0	0
Write 4 (R7 = 255)	11	0	0	25	0	100	0	255

Explanation: In each write only one register receives load = 1 from the DMux8Way, so only that register changes and the other seven keep their old values. In step 6 load = 0, so the in value 999 is ignored and R5 still holds 100. The read steps (5 to 7) do not change any register.

Test Script (RAM8.tst)

```
load RAM8.hdl,  
output-file RAM8.out,  
compare-to RAM8.cmp,  
output-list time%S1.4.1 in%D1.6.1 load%B2.1.2 address%D3.1.3 out%D1.6.1;
```

```
set in 11, set load 1, set address 0, tick, output; tock, output;  
set in 25, set load 1, set address 3, tick, output; tock, output;  
set in 100, set load 1, set address 5, tick, output; tock, output;  
set in 255, set load 1, set address 7, tick, output; tock, output;  
set in 0, set load 0, set address 3, tick, output; tock, output;  
set in 999, set load 0, set address 5, tick, output; tock, output;  
set in 0, set load 0, set address 1, tick, output; tock, output;
```